

Helm

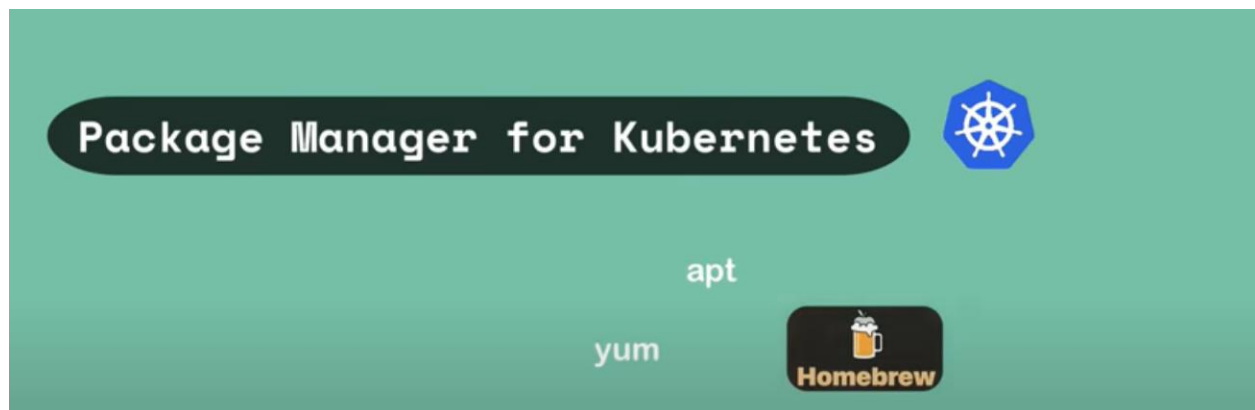
What is helm?

UC: We have a 100 Micro Services, each of them has config file, service. yaml how we manage as a single unit.

Helm is K8s package manager.



To package YAML files and distribute them in public and private repositories



UC: We have already deployed our application in K8s cluster, you want to deploy additionally, Elastic Search in your cluster to collect logs of your application, in order to deploy Elastic stack in your cluster you need additional resources e.g., StatefulSet (for stateful app), ConfigMap (for external configuration), secret (for secret data), K8s User with permissions and couple of services, manually it will take lots of time, if someone collects all these yaml file and package them and made them available somewhere by pushing in Helm Repo, that bundle of yaml files are called Helm Chart.

Helm Chart:

Group/ Collection of kubernetes resources (.yaml), put in to the pre-defined directory structure.

helm install myapp mychart/

helm upgrade myapp mychart/

helm uninstall myapp

mychart/ (chart name can be any)

Chart.yaml

templates/

deployment.yaml

service.yaml

pv.yaml

pvc.yaml

```
name: myapp
version: 1.0
description: A Helm Chart for simple app
~
~
```

```
root@master:~/mychart# ls -l
total 8
-rw-r--r-- 1 root root 82 Sep 14 16:11 Chart.yaml
drwxr-xr-x 2 root root 4096 Sep 14 16:09 templates
root@master:~/mychart# ls -l templates/
total 24
-rw-r--r-- 1 root root 495 Sep 14 16:09 deployment.yaml
-rw-r--r-- 1 root root 180 Sep 14 16:09 dev-db-conf.yaml
-rw-r--r-- 1 root root 319 Sep 14 16:09 pv.yaml
-rw-r--r-- 1 root root 182 Sep 14 16:09 pvc.yaml
-rw-r--r-- 1 root root 331 Sep 14 16:09 sec.yaml
-rw-r--r-- 1 root root 510 Sep 14 16:09 svc.yaml
root@master:~/mychart# cd
```

helm list

helm lint <CHART NAME>

helm repo list

helm repo add lerndevops <https://lerndevops.github.io/helm-charts/>

git repo list

helm search repo lerndevops

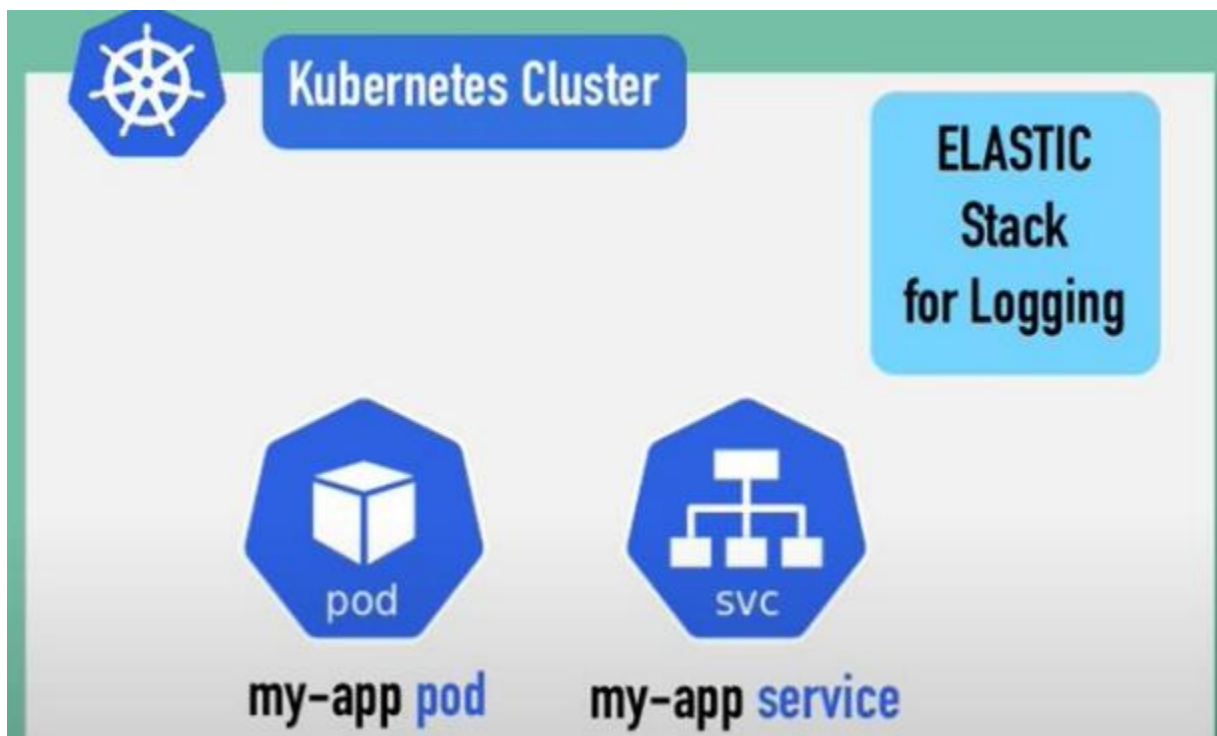
helm install springapp lerndevops/springbootapp

helm list

helm repo upgrade

helm uninstall springapp

- Bundle the yaml files
- Create your Own Helm Charts with helm
- Push them to Helm Repository
- Downloaded and use existing ones



Commonly Used Deployments: (Those Charts already available)

- Database applications
- Elastic Search
- MongoDB
- MySQL
- Monitoring App (Prometheus)

You can reuse the existing charts using the helm install <chart name> without additional efforts.

helm search <keyword> # search in helm hub

What we can do with helm chart?

- helm install <chart Name>
- helm upgrade <chart Name>
- helm delete <chart Name>

How to Install Helm?

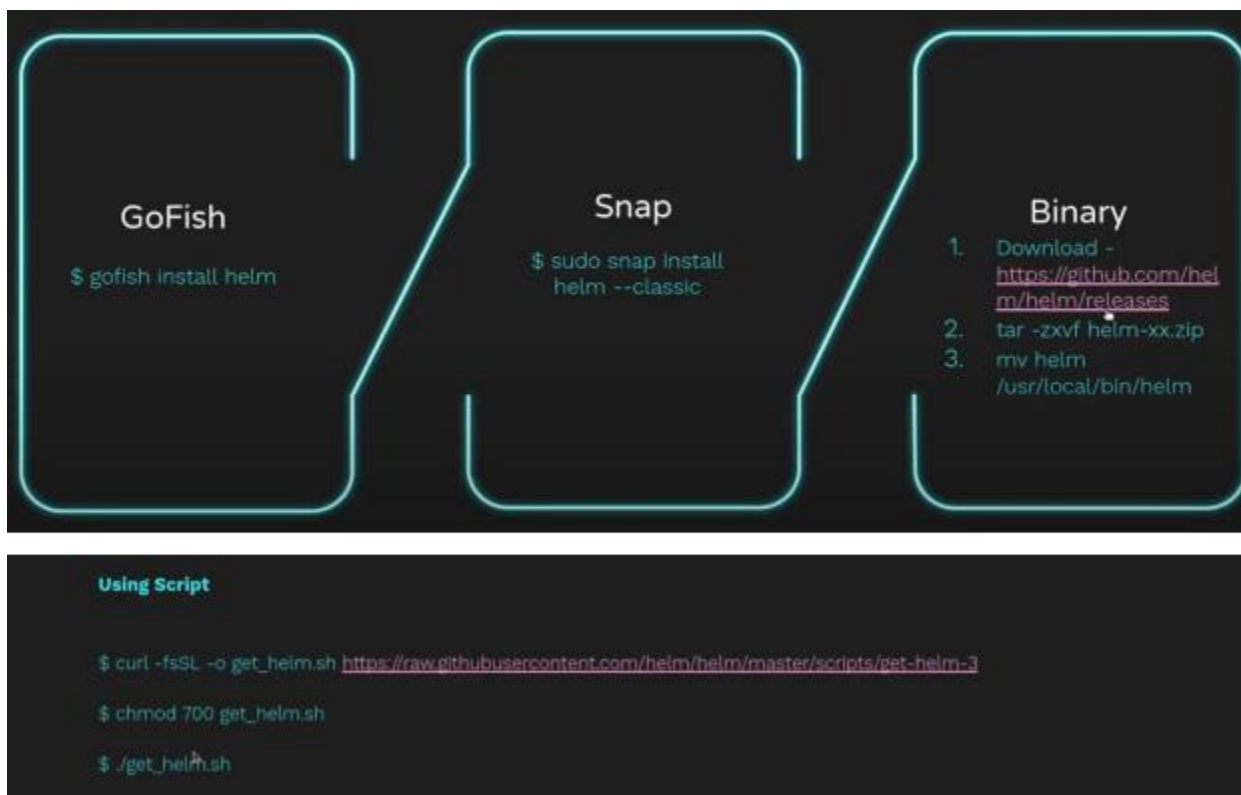
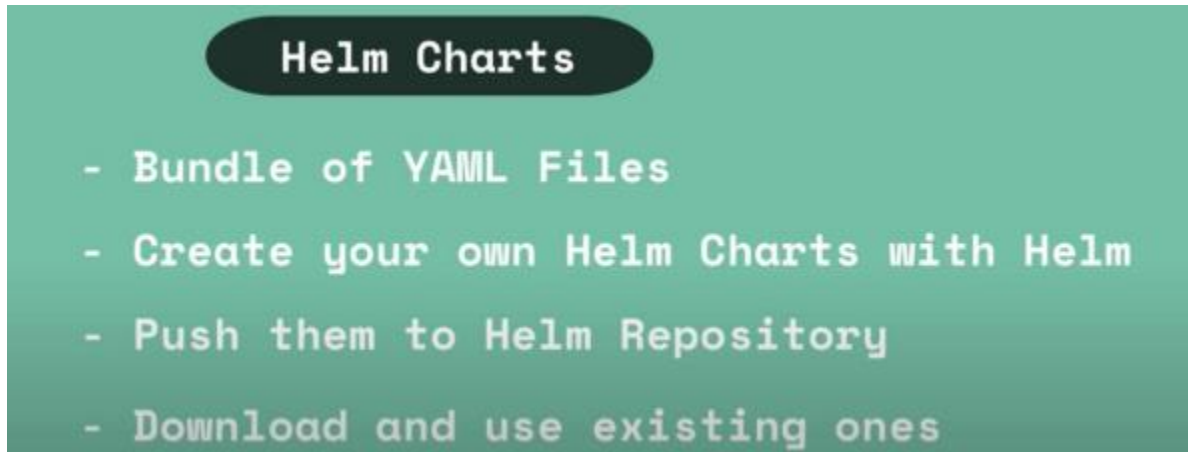


Chart: A Chart is a Helm package. It contains all of the resource definitions necessary to run an application, tool, or service inside of a Kubernetes cluster. Think of it like the Kubernetes equivalent of a Homebrew formula, an apt dpkg, or a Yum RPM

file.

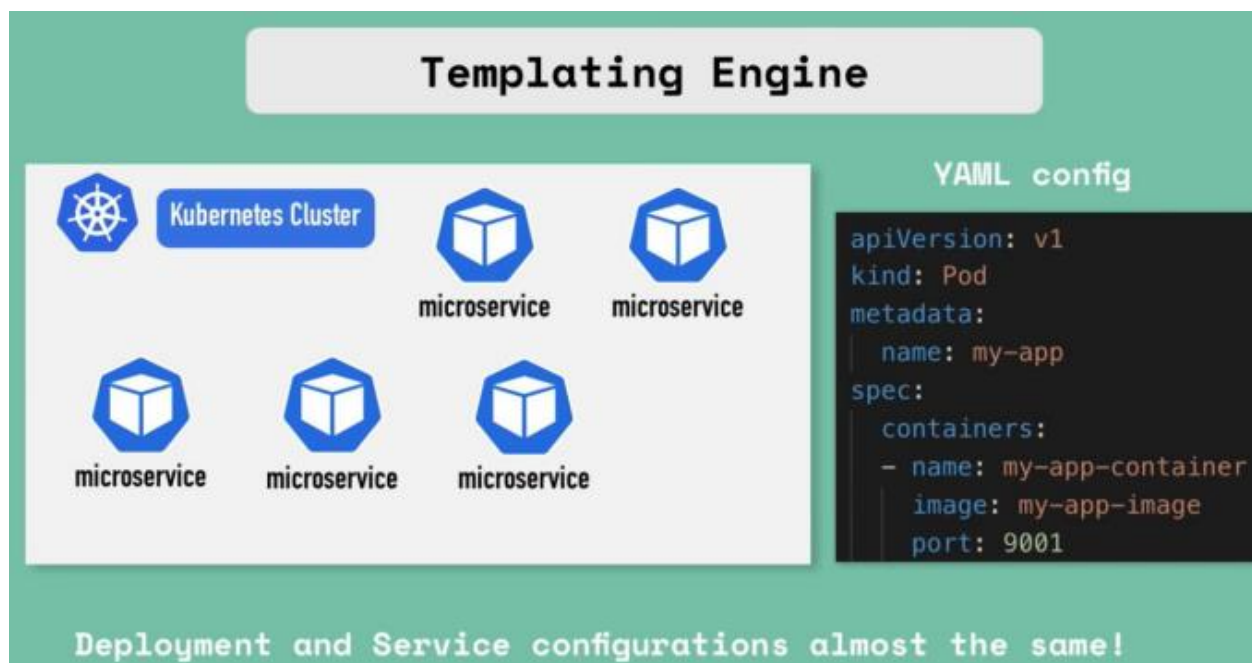
- Bundle of YAML files
- Create your own Helm Charts with Helm.

- Push them to Helm Repository
- Download and use existing ones
- Public Registries
- Private Registries

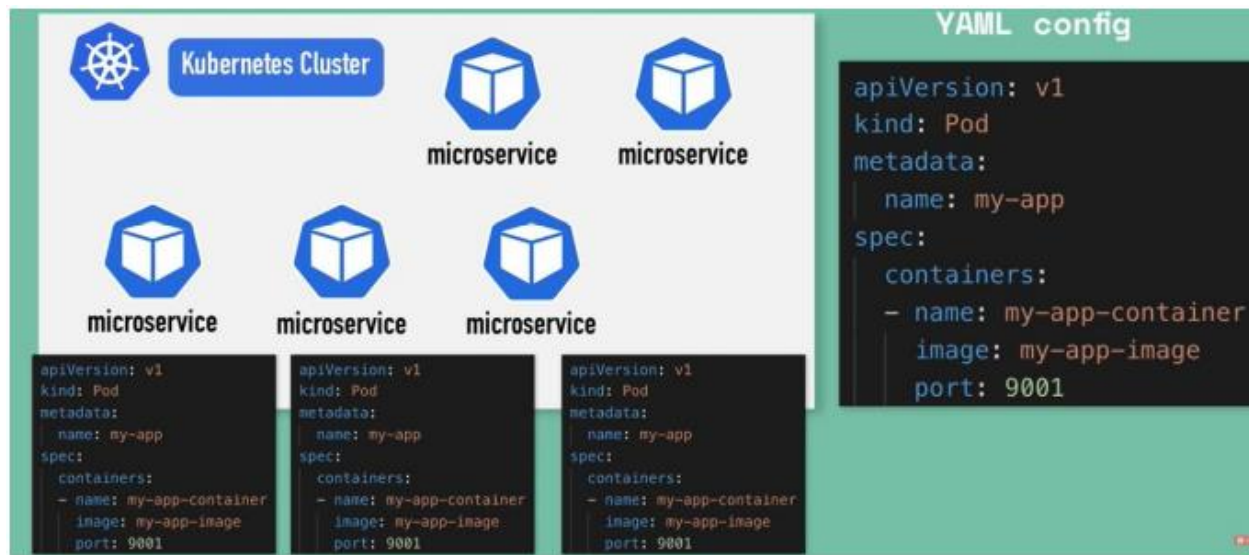


Another feature: Templating Engine

UC: You have an application made up of multiple Micro Services, we have to deploy in K8s cluster, all of them have deployment and service configuration almost the same, with only difference is application name and the version.



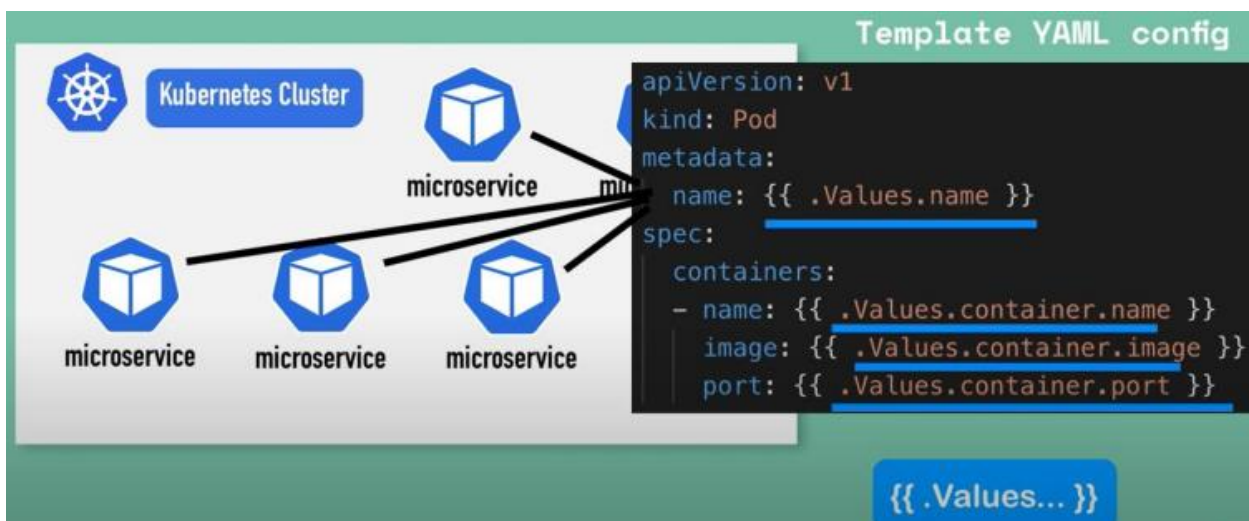
Without the helm, you have to write different yaml files for each of the deployment.



Since most of the values are same.

With the help of helm

- 1) Define a common blue print (That would be a template file)
- 2) Dynamic Values are replaced by placeholder



`{{ .Values }}` means you are taking values from external configuration (value. yaml), `.Value` is an object , which is created based on the values defined.

Values defined either via yaml file or with `-set` flag

values.yaml

```

name: my-app
container:
  name: my-app-container
  image: my-app-image
  port: 9001

```

Template YAML config

```

apiVersion: v1
kind: Pod
metadata:
  name: {{ .Values.name }}
spec:
  containers:
    - name: {{ .Values.container.name }}
      image: {{ .Values.container.image }}
      port: {{ .Values.container.port }}

```

Values defined either via yaml file or with --set flag

Now instead of many yaml files, you have one values are change dynamically.

many Yaml Files

```

apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
    - name: my-app-container
      image: my-app-image
      port: 9001

```

just 1 Yaml File

```

apiVersion: v1
kind: Pod
metadata:
  name: {{ .Values.name }}
spec:
  containers:
    - name: {{ .Values.container.name }}
      image: {{ .Values.container.image }}
      port: {{ .Values.container.port }}

```

Use in CI/CD pipeline:

Practical for CI / CD

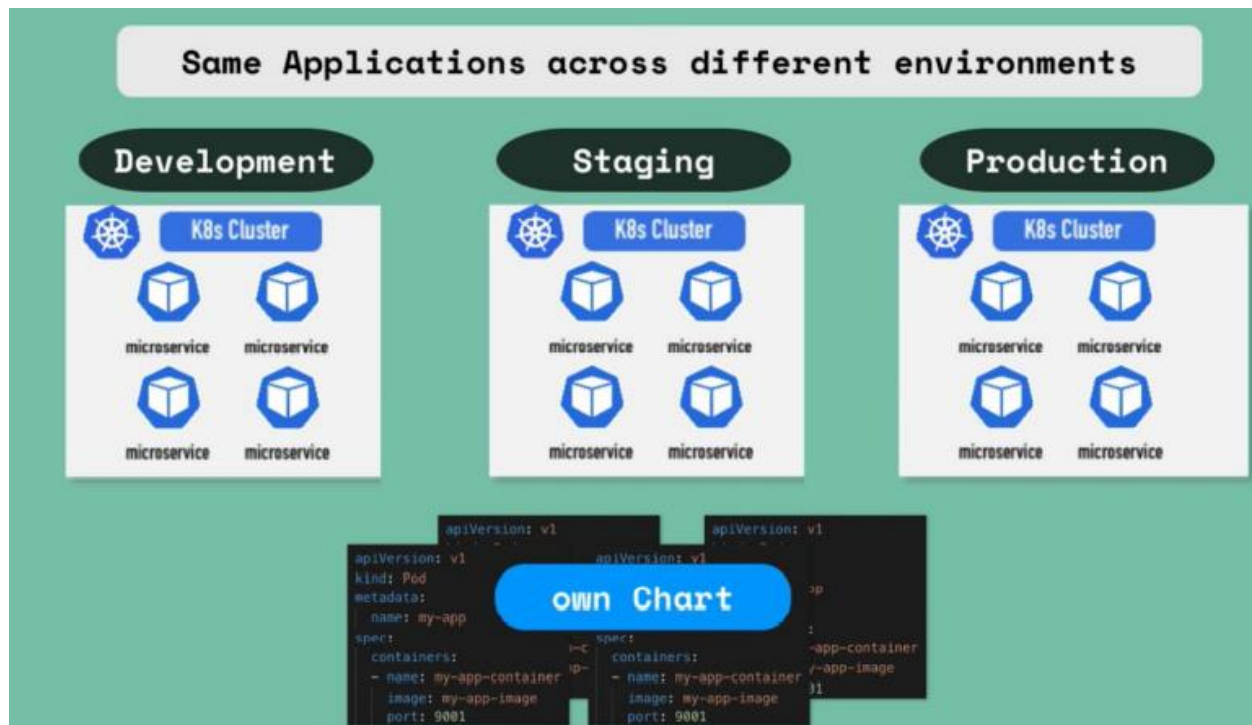
In your Build you can replace the values on the fly

```

apiVersion: v1
kind: Pod
metadata:
  name: {{ .Values.name }}
spec:
  containers:
    - name: {{ .Values.container.name }}
      image: {{ .Values.container.image }}
      port: {{ .Values.container.port }}

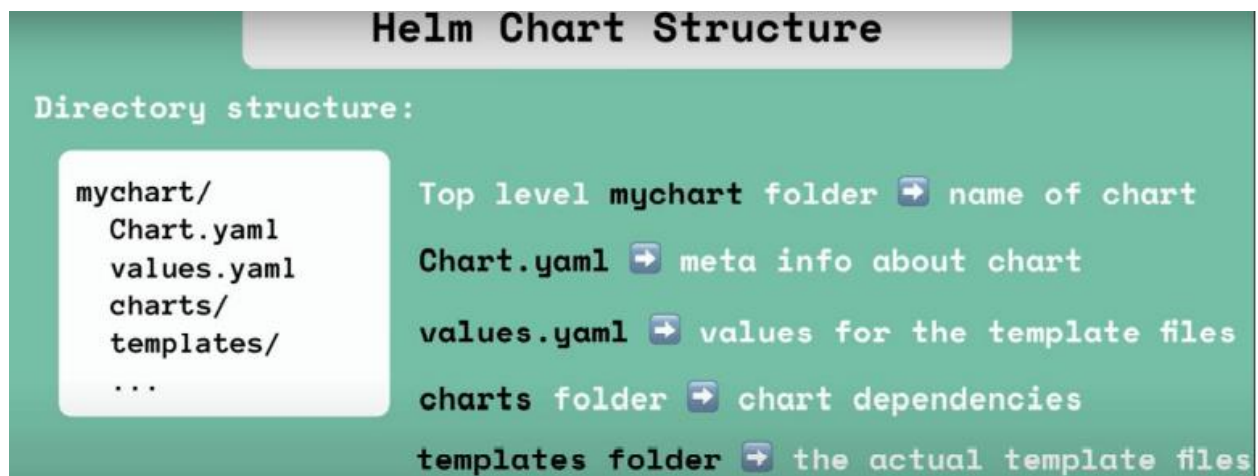
```

UC#3 Same application across different environments, across K8s Cluster, you have to deploy your microservice in different-different environments.



Instead of deploy separate files in each cluster, you package them to create your own helm chart, use the same to deploy in different environment.

Helm Chart Structure:




```
mychart/  
  Chart.yaml  
  values.yaml  
  charts/  
  templates/  
  ...
```

Template files will be filled with
the values from values.yaml

```
helm install <chartname>
```

values injected into the template file.

values.yaml

```
imageName: myapp  
port: 8080  
version: 1.0.0
```

default

my-values.yaml

```
version: 2.0.0
```

```
helm install --values=my-values.yaml <chartname>
```

values.yaml

```
imageName: myapp  
port: 8080  
version: 1.0.0
```

default

my-values.yaml

```
version: 2.0.0
```

override values

result

```
imageName: myapp  
port: 8080  
version: 2.0.0
```

.Values object

UC3# Release Management (Difference between v2 Vs. v3)

Helm Version 2 comes in two parts:



Keeping track of all chart executions:

Revision	Request
1	Installed chart
2	Upgraded to v 1.0.0

- Changes are applied to existing deployment instead of creating a new one

```
helm install <chartname>
```

```
helm upgrade <chartname>
```

- Tiller has too much power inside of K8s cluster

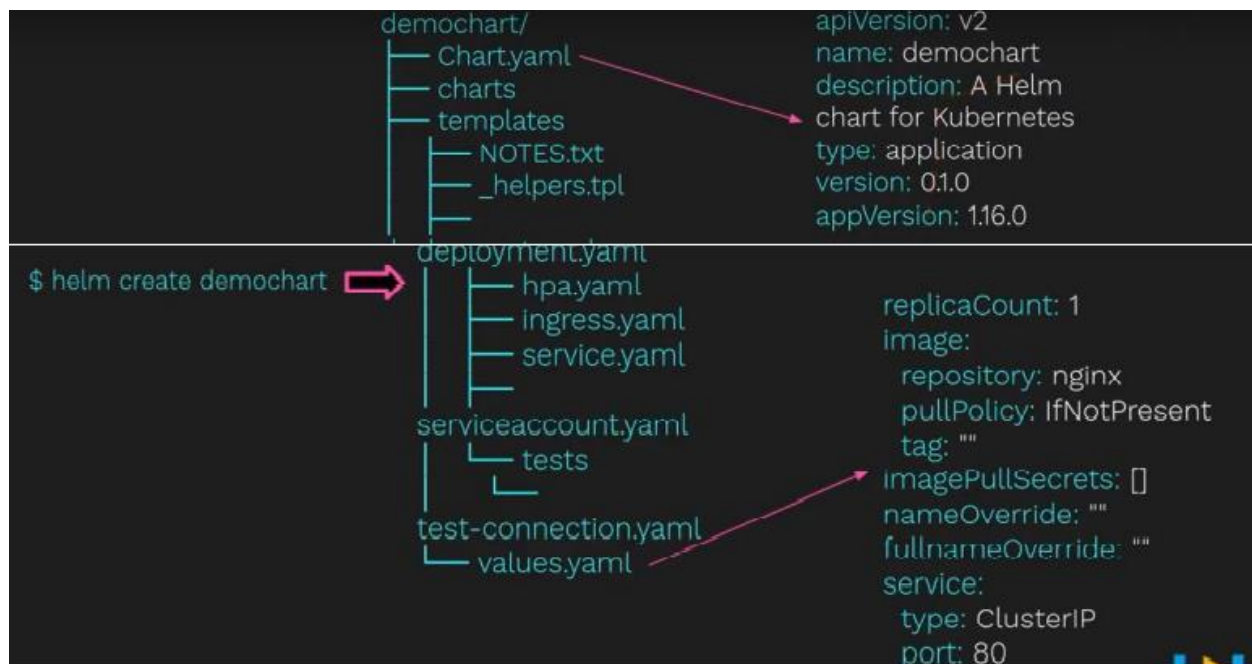
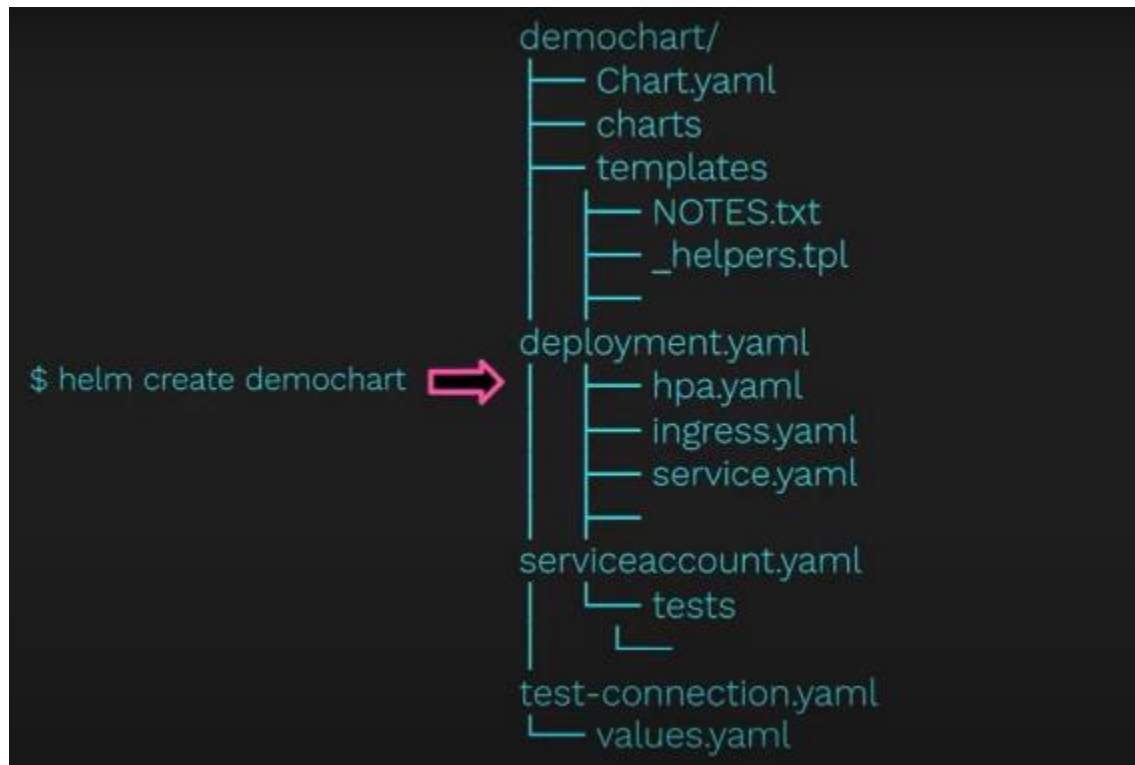
- Security Issue

In Helm 3 Tiller got removed!

- Solves the Security Concern, but makes it more difficult to use



helm create demochart



Difference between normal deployment and Helm deployment?

Normal Kubernetes deployment.yaml	Helm deployment.yaml
<pre>apiVersion: apps/v1 kind: Deployment metadata: name: RELEASE-NAME-springboot spec: replicas: 2 selector: matchLabels: app.kubernetes.io/name:.springboot</pre>	<pre>apiVersion: apps/v1 kind: Deployment metadata: name: RELEASE-NAME-springboot spec: replicas: {{.Values.replicaCount}} selector: matchLabels: app.kubernetes.io/name:.springboot</pre>

values.yaml →

```
replicaCount: 1
image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: ""
imagePullSecrets: []
nameOverride: ""
fullnameOverride: ""
service:
  type: ClusterIP
```

Day to Day Operations?

Normal Kubernetes	Helm
○ Setup kubernetes cluster ○ Create deployment.yaml ○ Create service.yaml ○ Create service-account.yaml	○ Setup kubernetes cluster ○ Create Helm chart ○ Fill in the values.yaml ○ Run - \$ helm install

Development Stage Production

\$ kubectl create deployment demochart --image=rahulwagh17/kubernetes:demochart

\$ kubectl expose deployment demochart --type=LoadBalancer --name=demo-service --external-ip=1111 --port=8080

Demo How to create Helm Chart?

step1: install helm

step2: kubectl get all

step3: helm create springboot

step4: tree springboot

step5: view actual yaml

cd ..

helm template springboot

step6: Identify possible issue

helm lint springboot # just like compilation

step7: dry-run

helm install <release name> --debug --dry-run <chartname>

step7 : helm install

helm install <release name> <chartname>

step8 : verify helm install

helm list -a

kubectl get all

How to upgrade helm release?

upgrade helm <release name> <chartname>

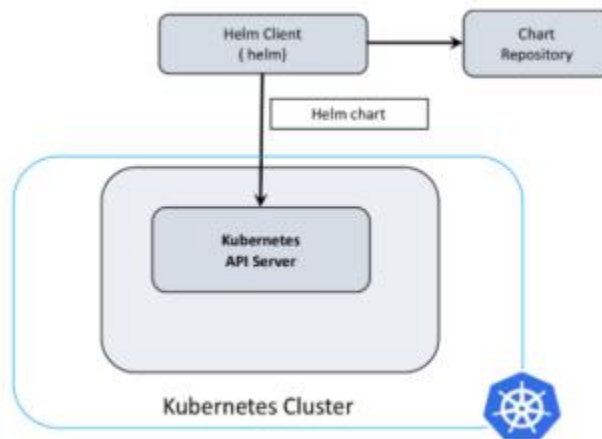
How to rollback helm release?

helm list -a

helm rollback <release name> < revision number>

Client Only Architecture

Helm 3 has a client-only architecture with the client still called helm. As seen in the following diagram, it operates similar to the Helm 2 client, but the client interacts directly with the Kubernetes API server. The in-cluster server Tiller is now removed.



It's not related to K8s, it's Add on top of k8s. Helm is package manager for your K8s, you have to install helm as separate package just like kubectl.

convert kubernetes.yaml to Helm chart yaml

